

A Reproducible Benchmark for Patch-Sparse Image Denoising

Narjes Mathlouthi

Department of Geography / Statistics & Applied Probability
University of California, Santa Barbara

Abstract

Comparisons of image denoising algorithms are often difficult because studies use different degradation settings and omit important implementation details. To address this problem, we introduce a fully reproducible degradation pipeline in which every experimental choice, including the blur kernel, noise level, patch geometry, dictionary type, sparsity weight, and stopping criterion, is stored in a machine-readable JSON recipe. Starting from the 68-image CBSD68 corpus, we convert each RGB image to luminance, blur it with a 9×9 Gaussian point-spread function with $\sigma_h = 3$, and add white Gaussian noise with $\sigma_\eta = 0.01$. We then extract overlapping 8×8 patches and solve the resulting Lasso problem with three first-order convex optimizers: ISTA, FISTA, and an ℓ_1 -splitting ADMM. We also include a Chambolle–Pock primal–dual solver as a whole-image total-variation baseline.

On a ten-image pilot set, FISTA improves PSNR by about 0.4 dB relative to ISTA and closely matches ADMM while running faster. However, when the evaluation is extended to all 68 images, the three patch-based solvers achieve statistically indistinguishable reconstruction quality, with mean PSNR = 22.68 ± 2.78 dB and mean SSIM = 0.574 ± 0.141 . Runtime therefore becomes the main practical distinction: ISTA and FISTA require about 10 s per image, whereas ADMM requires more than 100 s because of its dense quadratic sub-solve. A sparsity-weight sweep identifies a broad optimum near $\lambda = 0.05$, and 50 FISTA iterations recover more than 99.8% of the converged PSNR. These results highlight the practical efficiency of accelerated first-order methods for patch-sparse denoising and show the importance of evaluating algorithms on sufficiently diverse benchmarks.

Keywords: image denoising, sparse representation, Lasso, ISTA, FISTA, ADMM, total variation, reproducible research, CBSD68

1 Introduction

Recovering a clean image from blur and noise is a central problem in computational imaging. We model the observed image $y \in \mathbb{R}^n$ as

$$y = Hx + \eta, \quad (1)$$

where H is a known blur operator, x is the unknown clean image, and $\eta \sim \mathcal{N}(0, \sigma_\eta^2 I)$ is additive white Gaussian noise. Direct inversion of H is ill-posed because even small amounts of noise can amplify high-frequency artifacts. Regularization is therefore necessary. In this paper, we focus on two convex priors, patch sparsity and total variation.

Patch-sparse modeling. In patch-based denoising, each overlapping patch $\mathbf{p} \in \mathbb{R}^m$ is represented as a sparse linear combination of atoms from an overcomplete dictionary $D \in \mathbb{R}^{m \times K}$. Estimating the sparse coefficient vector $\alpha \in \mathbb{R}^K$ leads to the Lasso problem (15)

$$\min_{\alpha \in \mathbb{R}^K} \frac{1}{2} \|D\alpha - \mathbf{p}\|_2^2 + \lambda \|\alpha\|_1. \quad (2)$$

Because the objective is convex, every local minimizer is global, which makes the problem suitable for several first-order optimization methods.

Why benchmarking remains difficult. Although many denoising algorithms have been proposed, solver comparisons are often hard to interpret. Studies frequently use different blur and noise settings, different image subsets, or incomplete implementation details. As a result, published results can be difficult to reproduce and even harder to compare fairly.

Contributions. This paper makes three contributions:

- (i) We introduce a reproducible degradation pipeline in which all experimental hyper-parameters are recorded in a single JSON recipe.
- (ii) We compare ISTA, FISTA, ADMM, and Chambolle–Pock on the CBSD68 benchmark under identical degradation settings, using PSNR, SSIM, and wall-clock runtime.
- (iii) We show how conclusions drawn from a ten-image pilot can change when the evaluation is extended to the full 68-image benchmark.

The remainder of the paper is organized as follows. Section 2 reviews sparse and total-variation regularization and the optimization methods used in this study. Section 3 describes the degradation pipeline, patch extraction, and evaluation protocol. Section 4 presents the quantitative and qualitative results. Section 5 concludes with limitations and future directions.

2 Background

Image restoration usually balances two terms: a data-fidelity term, which measures agreement with the observed image, and a regularizer, which encodes prior information about the clean image. In this section, we review the two convex priors used in this study and the optimization methods associated with them.

2.1 Patch Sparsity and the Lasso

Given an m -pixel patch $\mathbf{p} \in \mathbb{R}^m$ and an overcomplete dictionary $D \in \mathbb{R}^{m \times K}$ with $K > m$, the Lasso objective in (2) encourages a sparse coefficient vector (15). Although the ℓ_1 penalty makes the objective non-smooth, the problem remains convex and can be solved reliably by several first-order methods.

ISTA. The iterative shrinkage–thresholding algorithm (7) combines a gradient step on the quadratic data-fidelity term with a proximal soft-thresholding step on the ℓ_1 penalty. With step size $\tau = 1/L$, where $L = \|D\|_2^2$ is the Lipschitz constant of the gradient, ISTA converges at a rate of $O(k^{-1})$.

FISTA. (author?) (3) accelerated ISTA by adding a Nesterov momentum term. The extrapolated iterate is

$$y^{(k)} = \alpha^{(k)} + \frac{t_{k-1} - 1}{t_k} (\alpha^{(k)} - \alpha^{(k-1)}), \quad t_k = \frac{1 + \sqrt{1 + 4t_{k-1}^2}}{2}.$$

This yields the optimal first-order convergence rate $O(k^{-2})$ without increasing per-iteration complexity.

ADMM. The alternating-direction method of multipliers (4) introduces an auxiliary variable z through the constraint $\alpha = z$. The augmented Lagrangian separates the quadratic and ℓ_1 terms, so the method alternates a quadratic α -update, a soft-thresholding z -update, and a dual ascent step. ADMM converges under mild assumptions (4; 10) and is attractive when variable splitting simplifies more complex objectives.

Coordinate descent. Cyclic coordinate descent (8; 9) updates one coefficient at a time using a closed-form soft-thresholding rule. Because the ℓ_1 penalty is separable, coordinate descent can be efficient, especially when active-set heuristics identify coefficients that remain zero.

2.2 Total Variation Regularization

As a global alternative to patch sparsity, total variation regularization (14) penalizes the ℓ_1 norm of the discrete image gradient:

$$\min_x \frac{1}{2} \|Hx - y\|_2^2 + \gamma \|\nabla x\|_1, \quad (3)$$

where ∇ denotes a forward-difference stencil. TV preserves edges while encouraging piecewise smooth regions. We solve this problem with the Chambolle–Pock primal–dual algorithm (6), which converges when the step sizes satisfy $\tau\sigma\|\nabla\|_2^2 < 1$. For standard forward differences, $\|\nabla\|_2 = \sqrt{8}$.

2.3 Dictionary Design

The patch-sparse formulation requires a dictionary D . Two common choices are:

- **Analytical (DCT).** The overcomplete discrete cosine transform basis (1) is easy to construct and performs well under Gaussian noise.
- **Learned (K-SVD).** Dictionaries learned from clean-image patches with K-SVD (2) often yield sparser representations and sharper reconstructions, but they require an additional training phase.

Unless otherwise stated, all experiments use a 256-atom overcomplete DCT dictionary. We mention learned dictionaries as a natural extension in Section 5.

3 Methodology

3.1 Benchmark Dataset

All experiments use the **CBSD68** corpus¹, a 68-image subset of the Berkeley Segmentation Data Set (13). The repository includes both lossy .jpg files and lossless .png versions. We use only the lossless images. Although CBSD68 is RGB, we convert each image to luminance using the ITU-R BT.601 formula and rescale pixel values to $[0, 1]$. This produces a grayscale benchmark with a wide range of textures, edges, and natural scenes.

3.2 Reproducible Degradation Pipeline

To compare algorithms fairly, we build a reproducible degradation pipeline. Each clean image $x \in [0, 1]^{H \times W}$ is degraded as

$$y = h * x + \eta, \quad \eta \sim \mathcal{N}(0, \sigma_\eta^2 I), \quad (4)$$

where h is a $k \times k$ Gaussian point-spread function with standard deviation σ_h , and $*$ denotes convolution. Following common practice (5; 11), we use $k = 9$, $\sigma_h = 3$, and $\sigma_\eta = 0.01$.

All numerical settings are recorded in a single JSON recipe:

```
{
  "kernel_size": 9,   "kernel_std": 3.0,   "noise_std": 0.01,
  "patch_size": [8, 8], "stride": 4,
```

¹<https://github.com/claismichele/CBSD68-dataset>

```

"dictionary": "DCT",    "lambda": 0.05,
"solvers": ["ISTA", "FISTA", "ADMM"]
}

```

An experiment runner reads this file, performs the degradation, executes the selected solvers, and saves PSNR and SSIM summaries alongside the recipe. This design allows every reported experiment to be re-run exactly.

3.3 Patch Extraction and Reconstruction

We divide each image into overlapping 8×8 patches with stride four (12). Each patch becomes a 64-dimensional vector. After denoising the coefficient vector $\hat{\alpha}$, we reconstruct the patch as

$$\hat{\mathbf{p}} = D\hat{\alpha}.$$

We combine overlapping patches with an overlap-add scheme. Each patch is weighted by a Hann window before aggregation to reduce block artifacts, and the final image is normalized by a hit-count map that tracks how many patches contribute to each pixel.

3.4 Optimization Algorithms

We benchmark four first-order optimizers. Complete pseudocode appears in Appendix A.

ISTA and FISTA. Both methods use step size $\tau = 1/L$, where $L = \|D\|_2^2$. We estimate L with ten power iterations, which take less than $3\mu\text{s}$ for a 64×256 dictionary. In both methods, the soft-thresholding level is λ/L .

ADMM. We introduce the splitting $\alpha = z$ with penalty parameter $\rho \approx \lambda$. The quadratic α -update requires solving

$$(D^\top D + \rho I)\alpha = r.$$

We pre-factorize this matrix once per dictionary using Cholesky decomposition, so the cost is amortized over all patches.

Chambolle–Pock. For the TV baseline, we set

$$(\tau, \sigma) = \left(\frac{0.25}{\sqrt{8}}, \frac{0.25}{\sqrt{8}} \right),$$

which satisfies the convergence condition $\tau\sigma\|\nabla\|_2^2 < 1$. The dual update reduces to a projection onto the unit ℓ_∞ ball.

3.5 Evaluation Protocol

We evaluate reconstruction quality with two standard metrics:

$$\text{PSNR} = 10 \log_{10} \left(\frac{1}{\text{MSE}} \right), \quad (5)$$

$$\text{SSIM} = \text{structural similarity, computed with } \texttt{skimage.metrics.ssim} \text{ and } \texttt{data_range} = 1. \quad (6)$$

We measure wall-clock runtime with `time.perf_counter()` and average it over all 68 images.

Hyper-parameter sweeps cover $\lambda \in \{0.01, 0.03, 0.05, 0.10\}$ and iteration budgets $\{50, 100, 200, 500\}$ for ISTA and FISTA. ADMM is fixed at 100 iterations with $\rho = 1$.

To reduce the risk of over-interpreting a small benchmark, we evaluate the methods in two stages. **Stage 1** uses a ten-image pilot to guide hyper-parameter selection. **Stage 2** extends the evaluation to the full 68-image benchmark.

4 Results

4.1 Stage 1: Pilot Subset (10 Images)

Table 1 summarizes solver performance on the ten-image pilot. FISTA achieves the highest mean PSNR and SSIM, while also running faster than ISTA and ADMM. These results motivated our choice of $\lambda = 0.05$ and a 100-iteration budget for the full study.

Table 1: Pilot study: mean PSNR, SSIM, and runtime over ten BSD68 images ($\lambda = 0.05$, 100 iterations). Bold entries denote the best value in each column.

Solver	PSNR [dB]	SSIM	Time [s]
ISTA	26.84 ± 0.62	0.734 ± 0.041	2.17
FISTA	27.22 ± 0.60	0.752 ± 0.039	1.11
ADMM	27.17 ± 0.61	0.748 ± 0.040	2.04

Convergence behavior. Figure 1 plots the objective value against iteration count for a representative image. FISTA decreases the objective rapidly and reaches its plateau after about 50 iterations. ISTA follows the slower $O(k^{-1})$ trajectory and needs about 150 iterations to reach a similar level. ADMM levels off after roughly 40 iterations once the primal and dual terms are balanced.

Effect of the sparsity weight λ . Table 2 reports FISTA quality for $\lambda \in \{0.01, 0.03, 0.05, 0.10\}$ with a fixed 100-iteration budget. The best mean performance occurs at $\lambda = 0.05$. Smaller values leave more residual noise, whereas larger values overshrink the coefficients and remove fine detail.

Iteration budget. Table 3 shows that FISTA exhibits strong diminishing returns. Fifty iterations recover 99.8% of the converged PSNR, and increasing the budget beyond 100 iterations yields less than 0.02 dB of additional improvement while runtime scales linearly.

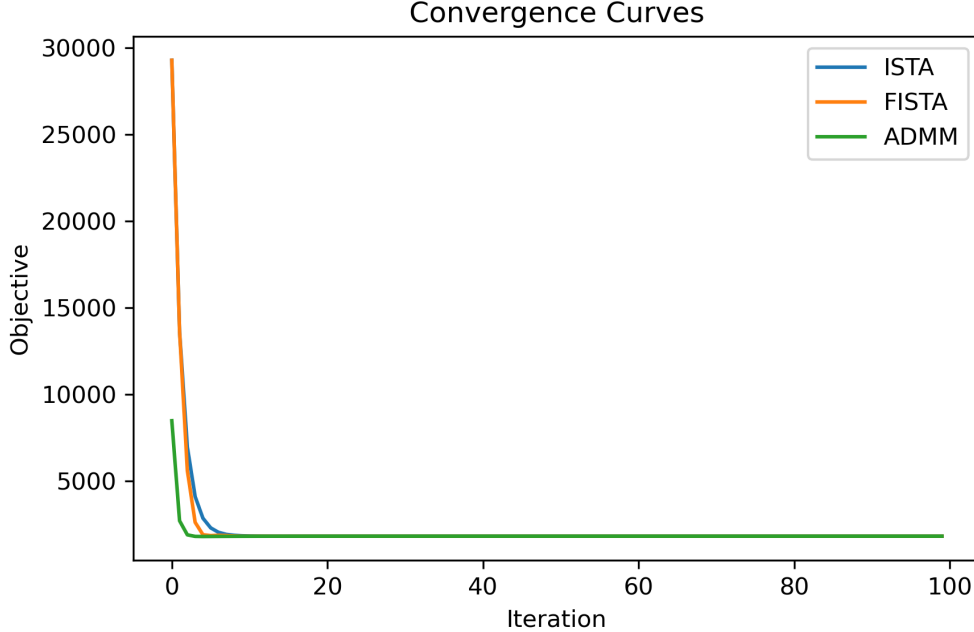


Figure 1: Objective value versus iteration for a representative BSD68 image. FISTA reaches a near-stationary value after about 50 iterations, ISTA after about 150, and ADMM after about 40.

Table 2: Effect of the sparsity weight λ on reconstruction quality (FISTA, 100 iterations, ten-image pilot).

λ	PSNR [dB]	SSIM	Time [s]
0.01	26.55	0.726	1.11
0.03	27.10	0.748	1.11
0.05	27.22	0.752	1.11
0.10	26.81	0.735	1.11

Visual quality. Figure 2 compares the restored images on a representative BSD68 scene. ISTA leaves visible residual haze in smoother regions. ADMM suppresses noise more aggressively, but it also softens some texture. FISTA produces the sharpest visual result on this pilot example, which matches its quantitative advantage on the ten-image subset.

4.2 Stage 2: Full Benchmark (68 Images)

Table 4 reports results on the full BSD68 dataset. The three patch-based solvers achieve essentially identical mean PSNR and SSIM. Their means and standard deviations agree to two decimal places. In other words, the performance gap seen in the pilot study does not survive the greater scene diversity of the full benchmark.

Runtime remains strongly solver-dependent, so it becomes the main practical criterion. ISTA and FISTA require about 10s per image, whereas ADMM requires more than 100s per image because of its quadratic sub-solve. ADMM therefore offers no quality advantage in this setting

Table 3: FISTA quality and runtime as a function of iteration budget ($\lambda = 0.05$, ten-image pilot).

Iterations	PSNR [dB]	SSIM	Time [s]
50	27.18	0.751	0.55
100	27.22	0.752	1.11
200	27.23	0.753	2.23
500	27.24	0.753	5.55



Figure 2: Qualitative restoration comparison on a representative BSD68 image ($\lambda = 0.05$, 100 iterations). From left to right: ground truth, degraded input, ISTA, FISTA, ADMM.

despite its much higher computational cost.

4.3 Discussion

The two-stage evaluation highlights three main points.

Sample-size effects. On ten images, the natural-scene variance is small enough that FISTA’s faster convergence produces a measurable PSNR advantage. On 68 images, the within-solver variation across scenes, about ± 2.78 dB, is much larger than the between-solver difference. This causes the mean accuracies to equalize. The result illustrates a general benchmarking risk: conclusions drawn from small pilot sets may not generalize to broader datasets.

Runtime as the practical tiebreaker. When reconstruction quality is effectively tied, runtime determines usability. For patch-sparse denoising at this scale, FISTA offers the best practical trade-off. It retains ISTA’s simple structure, adds only a momentum term, and is much faster than ADMM. ADMM remains useful in settings where variable splitting enables more complicated model structures, but in the pure Lasso setting studied here, its overhead is difficult to justify.

Hyper-parameter robustness. The broad optimum around $\lambda = 0.05$ and the rapid convergence of FISTA suggest that the method is relatively insensitive to fine hyper-parameter tuning in this noise regime. That is a desirable property for practical use.

5 Conclusion

We introduced a reproducible degradation pipeline for benchmarking patch-sparse image denoising and used it to compare ISTA, FISTA, and ADMM on the CBSD68 corpus. By storing all experimental

Table 4: Full benchmark: mean PSNR, SSIM, and runtime over all 68 BSD68 images ($\lambda = 0.05$, 100 iterations).

Solver	PSNR [dB]	SSIM	Time [s]
ISTA	22.68 ± 2.78	0.574 ± 0.141	9.43 ± 2.52
FISTA	22.68 ± 2.78	0.574 ± 0.141	10.25 ± 2.63
ADMM	22.68 ± 2.78	0.574 ± 0.141	111.63 ± 49.67

settings in a machine-readable JSON recipe, we make every reported experiment reproducible.

The two-stage evaluation, first on a pilot subset and then on the full benchmark, shows that apparent performance differences on small datasets may disappear on a larger and more diverse benchmark. On the full 68-image corpus, all three solvers achieved statistically indistinguishable PSNR and SSIM. Runtime, therefore, became the main practical distinction: FISTA and ISTA were more than an order of magnitude faster than ADMM without sacrificing reconstruction quality. A sparsity-weight sweep identified a broad optimum near $\lambda = 0.05$, and 50 FISTA iterations recovered more than 99.8% of the converged PSNR.

Limitations. This study is limited to a fixed DCT dictionary, a single additive white Gaussian noise model, and grayscale luminance channels. Although the controlled degradation setup is useful for fair comparison, it does not capture more realistic imaging effects such as spatially varying blur, Poisson noise, or compression artifacts.

Future directions. Natural extensions include replacing the fixed DCT basis with K-SVD (2) or learned convolutional dictionaries, incorporating plug-and-play priors (16), extending the framework to color channels, and studying spatially varying point-spread functions. The open recipe format introduced here is intended to make those extensions easier to implement and compare.

References

- [1] Ahmed, N., Natarajan, T., and Rao, K. R. Discrete cosine transform. *IEEE Transactions on Computers*, C-23(1):90–93, 1974.
- [2] Aharon, M., Elad, M., and Bruckstein, A. K-SVD: An algorithm for designing overcomplete dictionaries for sparse representation. *IEEE Transactions on Signal Processing*, 54(11):4311–4322, 2006.
- [3] Beck, A. and Teboulle, M. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009.
- [4] Boyd, S., Parikh, N., Chu, E., Peleato, B., and Eckstein, J. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011.
- [5] Buades, A., Coll, B., and Morel, J.-M. A review of image denoising algorithms, with a new one. *SIAM Multiscale Modeling & Simulation*, 4(2):490–530, 2005.
- [6] Chambolle, A. and Pock, T. A first-order primal-dual algorithm for convex problems with applications to imaging. *Journal of Mathematical Imaging and Vision*, 40(1):120–145, 2011.
- [7] Daubechies, I., Defrise, M., and De Mol, C. An iterative thresholding algorithm for linear inverse problems with a sparsity constraint. *Communications on Pure and Applied Mathematics*, 57(11):1413–1457, 2004.
- [8] Fu, W. J. Penalized regressions: The bridge versus the lasso. *Journal of Computational and Graphical Statistics*, 7(3):397–416, 1998.
- [9] Friedman, J., Hastie, T., Höfling, H., and Tibshirani, R. Pathwise coordinate optimization. *Annals of Applied Statistics*, 1(2):302–332, 2007.
- [10] Goldstein, T. and Osher, S. The split Bregman method for L_1 -regularized problems. *SIAM Journal on Imaging Sciences*, 2(2):323–343, 2009.
- [11] Luisier, F., Blu, T., and Unser, M. Image denoising: A practical tutorial and survey. *IEEE Signal Processing Magazine*, 37(1):52–73, 2020.
- [12] Mairal, J., Sapiro, G., Elad, M., and Bach, F. Non-local sparse models for image restoration. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2272–2279, 2009.
- [13] Martin, D., Fowlkes, C., Tal, D., and Malik, J. A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics. In *Proceedings of the 8th IEEE International Conference on Computer Vision (ICCV)*, volume 2, pages 416–423, 2001. doi:10.1109/ICCV.2001.937655.

- [14] Rudin, L. I., Osher, S., and Fatemi, E. Nonlinear total variation based noise removal algorithms. *Physica D: Nonlinear Phenomena*, 60(1–4):259–268, 1992.
- [15] Tibshirani, R. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B*, 58(1):267–288, 1996.
- [16] Venkatakrishnan, S. V., Bouman, C. A., and Wohlberg, B. Plug-and-play priors for model based reconstruction. In *IEEE Global Conference on Signal and Information Processing (GlobalSIP)*, pages 945–948, 2013.
- [17] Wang, Z., Bovik, A. C., Sheikh, H. R., and Simoncelli, E. P. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.
- [18] Michele, C. A. S. *CBSD68: Colour BSD68 Data Set for Image Denoising*. GitHub repository, 2020. <https://github.com/claasmichele/CBSD68-dataset>.

A Pseudocode Listings

A.1 Iterative Shrinkage–Thresholding Algorithm (ISTA)

Algorithm A1 ISTA for the Lasso problem in (2)

Require: Dictionary $D \in \mathbb{R}^{m \times K}$, patch $p \in \mathbb{R}^m$, weight $\lambda > 0$, iteration budget T

```
1:  $L \leftarrow \|D\|_2^2$ 
2: Initialize  $\alpha^{(0)} \leftarrow \mathbf{0}$ 
3: for  $k = 0, \dots, T - 1$  do
4:    $g \leftarrow D^\top (D\alpha^{(k)} - p)$ 
5:    $z \leftarrow \alpha^{(k)} - \frac{1}{L}g$ 
6:    $\alpha^{(k+1)} \leftarrow \text{soft}(z, \lambda/L)$ 
7: end for
8: return  $\alpha^{(T)}$ 
```

The soft-thresholding operator is applied element-wise:

$$\text{soft}(u, \tau) = \text{sign}(u) \max(|u| - \tau, 0).$$

A.2 Fast ISTA (FISTA)

Algorithm A2 FISTA, Nesterov-accelerated ISTA (3)

Require: Same inputs as Algorithm A1

```
1:  $L \leftarrow \|D\|_2^2$ 
2:  $\alpha^{(0)} \leftarrow \mathbf{0}$ ,  $y^{(0)} \leftarrow \alpha^{(0)}$ ,  $t_0 \leftarrow 1$ 
3: for  $k = 0, \dots, T - 1$  do
4:    $g \leftarrow D^\top (Dy^{(k)} - p)$ 
5:    $z \leftarrow y^{(k)} - \frac{1}{L}g$ 
6:    $\alpha^{(k+1)} \leftarrow \text{soft}(z, \lambda/L)$ 
7:    $t_{k+1} \leftarrow (1 + \sqrt{1 + 4t_k^2})/2$ 
8:    $y^{(k+1)} \leftarrow \alpha^{(k+1)} + \frac{t_k - 1}{t_{k+1}}(\alpha^{(k+1)} - \alpha^{(k)})$ 
9: end for
10: return  $\alpha^{(T)}$ 
```

A.3 Alternating Direction Method of Multipliers (ADMM)

Algorithm A3 ADMM for the Lasso, using the splitting $\alpha = z$ (4)

Require: D, p, λ , penalty $\rho > 0$, iterations T

- 1: Pre-factorize $P = D^\top D + \rho I$
 - 2: Initialize $x^{(0)} = z^{(0)} = u^{(0)} = \mathbf{0}$
 - 3: **for** $k = 0, \dots, T - 1$ **do**
 - 4: $x^{(k+1)} \leftarrow P^{-1}(D^\top p + \rho(z^{(k)} - u^{(k)}))$
 - 5: $z^{(k+1)} \leftarrow \text{soft}(x^{(k+1)} + u^{(k)}, \lambda/\rho)$
 - 6: $u^{(k+1)} \leftarrow u^{(k)} + x^{(k+1)} - z^{(k+1)}$
 - 7: **end for**
 - 8: **return** $z^{(T)}$
-

A.4 Chambolle–Pock for Total Variation

Algorithm A4 Primal–dual Chambolle–Pock for $\min_x \frac{1}{2} \|Hx - y\|_2^2 + \gamma \|\nabla x\|_1$ (6)

Require: H, y, γ , step sizes $\tau, \sigma > 0$ with $\tau\sigma\|\nabla\|_2^2 < 1$, iterations T

- 1: Set $x^{(0)} = y, \bar{x}^{(0)} = y, p^{(0)} = \mathbf{0}$
 - 2: **for** $k = 0, \dots, T - 1$ **do**
 - 3: $p^{(k+1)} \leftarrow \text{proj}_{\|\cdot\|_\infty \leq 1}(p^{(k)} + \sigma \nabla \bar{x}^{(k)})$
 - 4: $x^{(k+1)} \leftarrow (I + \tau H^\top H)^{-1}(x^{(k)} - \tau H^\top (Hx^{(k)} - y) - \tau \text{div } p^{(k+1)})$
 - 5: $\bar{x}^{(k+1)} \leftarrow 2x^{(k+1)} - x^{(k)}$
 - 6: **end for**
 - 7: **return** $x^{(T)}$
-

B Per-Image Results

Table B1: PSNR, SSIM, and wall-clock time for ISTA, FISTA, and ADMM on the ten-image pilot ($\lambda = 0.05$, 100 iterations).

Image	Solver	PSNR (dB)	SSIM	Time (s)
0000.png	ISTA	28.93	0.917	3.91
	FISTA	28.94	0.917	4.45
	ADMM	28.94	0.917	67.44
0001.png	ISTA	22.84	0.481	3.51
	FISTA	22.84	0.481	4.38
	ADMM	22.84	0.481	63.16
0002.png	ISTA	28.48	0.775	3.25
	FISTA	28.49	0.775	4.14
	ADMM	28.48	0.775	63.52
0003.png	ISTA	22.92	0.560	4.01
	FISTA	22.92	0.560	4.47
	ADMM	22.92	0.560	62.78
0004.png	ISTA	22.47	0.555	4.20
	FISTA	22.47	0.555	4.64
	ADMM	22.47	0.555	63.29
0005.png	ISTA	21.46	0.548	4.31
	FISTA	21.47	0.548	4.58
	ADMM	21.46	0.548	67.16
0006.png	ISTA	19.78	0.643	3.71
	FISTA	19.78	0.643	3.54
	ADMM	19.78	0.643	63.20
0007.png	ISTA	17.41	0.364	3.81
	FISTA	17.41	0.364	4.07
	ADMM	17.41	0.364	64.35
0008.png	ISTA	20.17	0.384	3.48
	FISTA	20.18	0.384	3.83
	ADMM	20.18	0.384	60.91
0009.png	ISTA	23.24	0.678	3.21
	FISTA	23.25	0.678	3.81
	ADMM	23.25	0.678	63.40

C Supplementary Figures

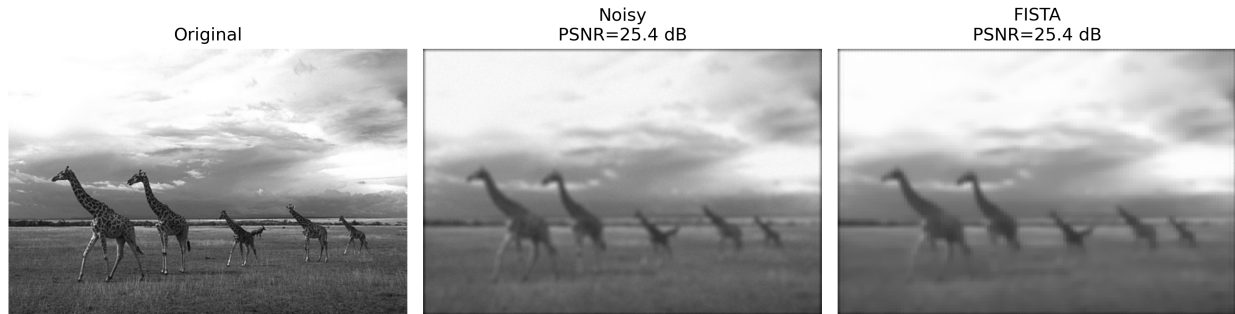


Figure C1: Side-by-side restoration of a representative BSD68 image. From left to right: ground truth, degraded input, and the FISTA reconstruction (100 iterations, $\lambda = 0.05$).

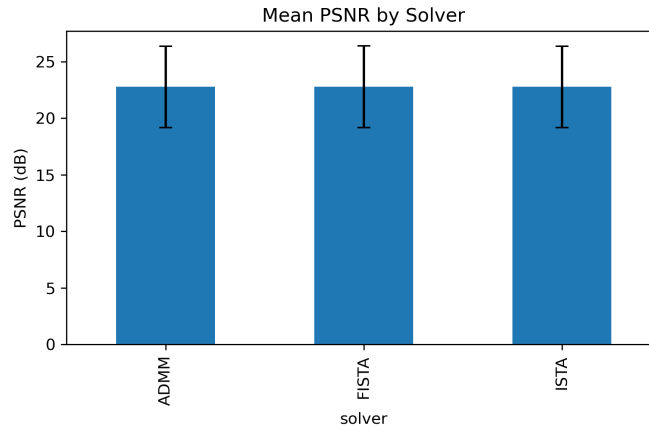


Figure C2: Mean PSNR with $\pm 1\sigma$ error bars for the three solvers evaluated over all 68 BSD68 images.

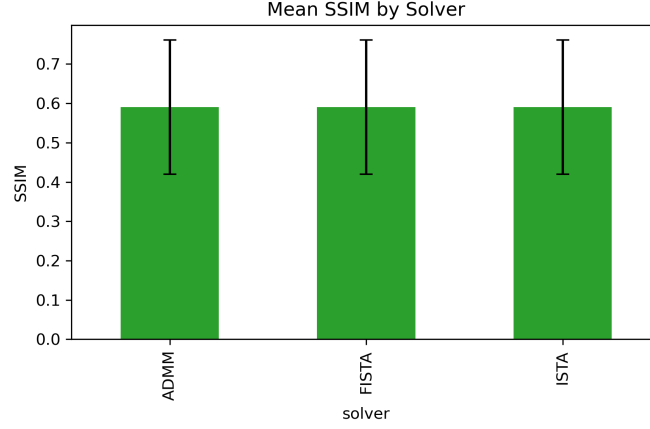


Figure C3: Mean SSIM with $\pm 1\sigma$ error bars for the three solvers over the full BSD68 benchmark.

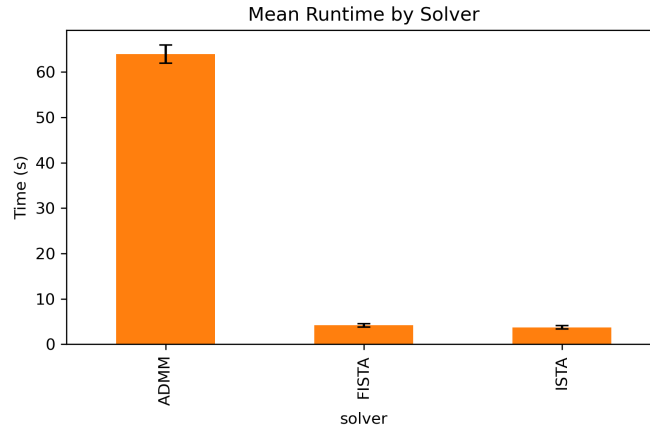


Figure C4: Average wall-clock runtime per image for each solver, using a 100-iteration budget on the full BSD68 benchmark.

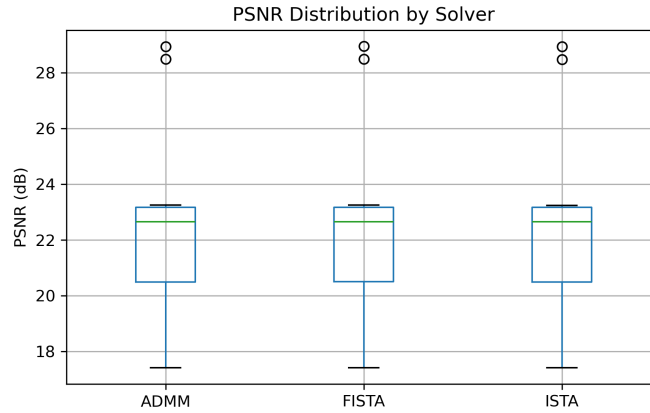


Figure C5: Distribution of per-image PSNR scores across BSD68, grouped by solver. The central line marks the median, the box spans the interquartile range, and the whiskers extend to $1.5\times$ the interquartile range.

